Proyectos con RTOS

Ejemplos de uso de RTOS con periféricos para IntelFPGA

mpeiro@eln.upv.es

Ejemplos de Periféricos

- Proyecto 1
  - PS2 Keyboard
  - SD Card
- Proyecto 2
  - Audio
- Proyecto 3
  - Puerto Serie UART

Inicio de estructuras

- MailBox
  - ISR_SW
- Flag de eventos
  - ISR_KEY
- Queue
  - ISR_Keyboard

```
MSG_box = OSMboxCreate(NULL);

INT8U ErrorFlags=OS_NO_ERR;
flagImage = OSFlagCreate(0x00, &ErrorFlags);
alt_ucosii_check_return_code(ErrorFlags);

MSG_queue = OSQCreate(&msgqueueTbl[0], MSG_QUEUE_SIZE);
```

ISR_ KEYs

```
/**
 * Subrutina de manejo de interrupcion por pulsador incrustada en uCOS
 */
void pushbutton_isr() {
OSIntEnter();
    int KEY_value = *(KEY_ptr + 3);         // Leemos registro de pulsadores
    *(KEY_ptr + 3) = 0;                     // Borrar la interrupcion

    if (KEY_value == 0x4) {
        printf("KEY2\n");
        INT8U Error=OS_NO_ERR;
        OSFlagPost(flagImage,(INT8U )0x1,OS_FLAG_SET,&Error);
        alt_ucosii_check_return_code(Error);
    }
    else if (KEY_value == 0x2) {
        printf("KEY1\n");
        int dma_salida = *((int*)DMA_SALIDA_EFECTOS_BASE + 0x3);
        dma_salida = dma_salida ^ 0x4;
        *((int*)DMA_SALIDA_EFECTOS_BASE + 0x3) =  dma_salida;
/*---------------------------- resto de opciones ----------------------------*/
    }
OSIntExit();
}
```

- Posteo en Flag la pulsación de un KEY.
- Usado para indicar a otra tarea que se solicita guardar una imagen en la SD card si KEY2.
- No finaliza else ☹

Task que atiende al FlagPost

- La tarea guarda una imagen en la tarjeta SD
- Cuando se pulsa KEY2
- Queda Pend de un POST
- POST el valor a 0

```
void task_save_image(){
    INT8U error_code = OS_NO_ERR;
    while(1){
        OSFlagPend(flagImage,0x1,OS_FLAG_WAIT_SET_ANY,0,&error_code);
        printf("Flag\n");
        alt_ucosii_check_return_code(error_code);
        OSFlagPost(flagImage,(INT8U )0x1,OS_FLAG_CLR,&error_code);
        alt_ucosii_check_return_code(error_code);
/* inicio de acciones de la task */
        alt_up_sd_card_dev *device_reference = NULL;
/* mas acciones sobre tarjeta sd….*/
```

ISR_Switch

```
void switch_isr(){
OSIntEnter();
    int SW_value = *(SW_switch_ptr);
    SW_photo = SW_value & 0xf;
    OSMboxPost(MSG_box, (void*)&SW_photo);

    if((SW_value>>4 & 1) == 1){
        printf("Imagen\n");
        *((int*)DMA_ENTRADA_EFECTOS_BASE + 1) = TARJETA_BASE;
        *((int*)DMA_ENTRADA_EFECTOS_BASE) = 0x01902;
    }else{
        printf("Video\n");
        *((int*)DMA_ENTRADA_EFECTOS_BASE + 1) = CAMERA_BASE;
        *((int*)DMA_ENTRADA_EFECTOS_BASE) = 0x123689;
    }

    SW_value = (SW_value>>5)&0xff;
    printf("%x\n", SW_value);
    *((int*)VIDEO_EFFECTS_0_BASE) = SW_value;
    printf("%x\n", *((int*)VIDEO_EFFECTS_0_BASE));

    *(SW_switch_ptr + 3) = 0;   // borra la interrupcion
OSIntExit();
}
```

- Uso de MailBox para indicar qué valor de foto se selecciona (SW[0..3]).
- Uso de SW4 para indicar si lectura de imagen de SD o Cámara.
- Uso de SW5 a SW12 para aplicar efectos de video

PS2_Keyboard



PS/2 Core for Intel DE-Series Boards

For Quartus Prime 17.0

- IP Para conectar teclado o ratón PS2, conexión serie a periférico externo y Memory Mapped a NIOS.
- FIFO 256 words
- Almacena datos recibidos PS2

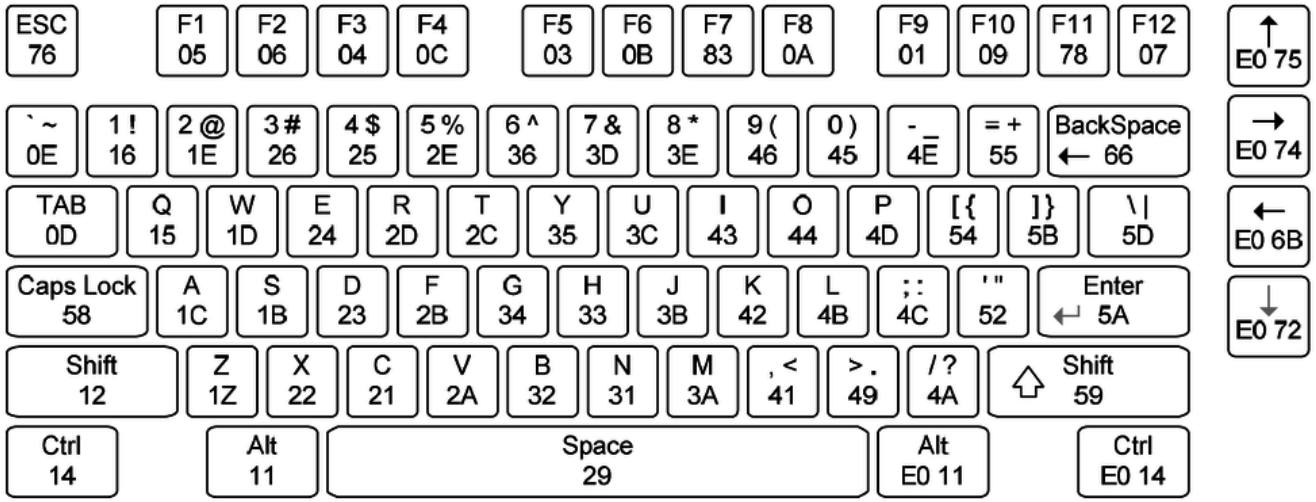
Table 1. PS/2 Core register map

Offset in bytes	Register Name	R/W/C	Bit description							
			31...16	15	14...11	10	9	8	7...1	0
0	data	R/W	RAVAIL	RVALID	*(1)*				DATA	
4	control	R/C	*(1)*			CE	*(1)*	RI	*(1)*	RE

PS2 Keyboard

- Tienen un código de rastreo (código MAKE) de 1 o 2 bytes cuando se pulsa una tecla
  - A: 1C, 1:16
- Cuando se libera la tecla (*release*) se envían dos códigos, primero F0 seguido del de rastreo (código BREAK)

KEY	MAKE	BREAK
A	1C	F0,1C
B	32	F0,32
C	21	F0,21
D	23	F0,23
E	24	F0,24
F	2B	F0,2B
G	34	F0,34
H	33	F0,33



Registros del IP PS2

- Registro de datos
- Registro de control
- RE -> Bit de interrupciones

Table 2. data register bits

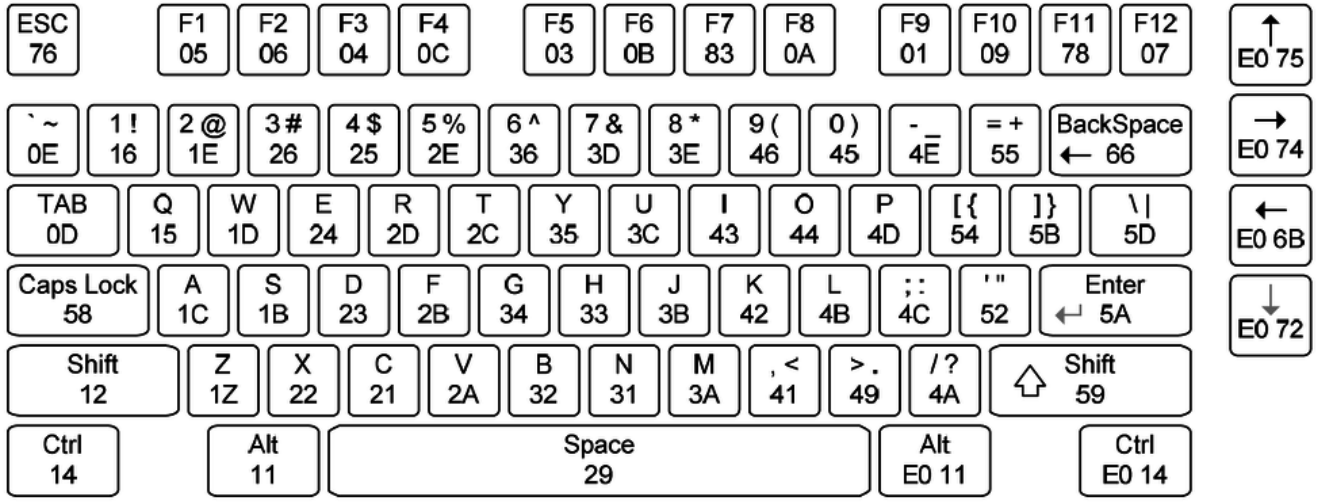
Bit number	Bit name	Read/Write/Clear	Description
7...0	DATA	R/W	The value to transfer to/from the PS/2 core. When writing, the DATA field is interpreted as a command to be sent to the PS/2 device. When reading, the DATA field is data from the PS/2 device.
15	RVALID	R	Indicates whether the DATA field is valid. If RVALID=1, then the DATA field is valid, else the DATA is undefined.
31...16	RAVAIL	R	The number of data items remaining in the read FIFO (including this read).

Table 3. control register bits

Bit number	Bit name	Read/Write/Clear	Description
0	RE	R/W	Interrupt-enable bit for read interrupts.
8	RI	R	Indicates that a read interrupt is pending.
10	CE	C	Indicates that an error occurred while trying to send a command to a PS/2 device.

Ejemplo sencillo de uso

```
volatile int * PS2_ptr = (int *)PS2_BASE;
int PS2_data, RVALID;
PS2_data = *(PS2_ptr);                    // read the Data register in the PS/2 port
RVALID = PS2_data & 0x8000;               // extract the RVALID field
if (RVALID) {
            …. keycode = PS2_data & 0xFF;
```



Habilitar interrupciones

- Se activan interrupciones de SW, KEY y Teclado PS2.
- La interrupción de teclado escribe un 1 en el bit RE (bit0) de la palabra de control.
- Para usar el HAL del PS2 se debería incluir
- altera_up_ps2_keyboard.h
- altera_up_ps2_mouse.h

```
#include "..\inc\irq_config.h"
#include "..\inc\isr.h"
#include "altera_up_avalon_ps2.h"

void CreateIRQ(){
    /* Enable switches interruption */
    *(SW_switch_ptr + 2) = 0xffff;
    *(SW_switch_ptr + 3) = 0;
    alt_irq_register(SWITCHES_IRQ, NULL, switch_isr);

    *(KEY_ptr + 2) = 0xff;
    *(KEY_ptr + 3) = 0;
    alt_irq_register(PUSHBUTTONS_IRQ, NULL, pushbutton_isr);

    *((int*)PS2_KEY_BASE+1) = 1;
    keyboardRelease = 0;
    alt_irq_register(PS2_KEY_IRQ, NULL, keyboard_isr);
}
```

ISR_Keyboard_PS2

- Lectura de la tecla pulsada por interrupciones.
- Si ENTER [5A] Se envía un -1

```
void keyboard_isr(){
OSIntEnter();
    alt_irq_disable(PS2_KEY_IRQ);
    int reg0 = *((int*)PS2_KEY_BASE);
    int keycode = reg0 & 0x000000FF;
    if(!keyboardRelease) {
        switch(keycode) {
            case 0x45: keyboardInput = 0; break;
            case 0x16: keyboardInput = 1; break;
            case 0x1e: keyboardInput = 2; break;
            case 0x26: keyboardInput = 3; break;
            case 0x25: keyboardInput = 4; break;
            case 0x2e: keyboardInput = 5; break;
            case 0x36: keyboardInput = 6; break;
            case 0x3d: keyboardInput = 7; break;
            case 0x3e: keyboardInput = 8; break;
            case 0x46: keyboardInput = 9; break;
            case 0x1c: keyboardInput = 0xa; break;
            case 0x32: keyboardInput = 0xb; break;
            case 0x21: keyboardInput = 0xc; break;
            case 0x23: keyboardInput = 0xd; break;
            case 0x24: keyboardInput = 0xe; break;
            case 0x2b: keyboardInput = 0xf; break;
            case 0x5a: keyboardInput = -1; break;
            case 0xf0: keyboardInput = -3; keyboardRelease = 1; break;
            default: keyboardInput = -2; break;
        }
    }
    else {
        keyboardRelease = 0;
    }
    printf("%d\n", keycode);
    if(!(keyboardInput < -1)){
        OSQPost(MSG_queue, (void*)&keyboardInput);
    }

    alt_irq_enable(PS2_KEY_IRQ);
OSIntExit();
}
```

Task de lectura del teclado

- Pendiente de cola
- Cada vez que se introduce un número se almacena en el reg1
- Se acumulan las cifras hasta alcanzar el ENTER(-1)
- La cifra de N números almacenados desde la cola se usa para selección de efectos de video
- No hay control de número máximo de código ni su tamaño

```
void task_read_keyboard(void* pdata){
    INT8U error_code = OS_NO_ERR;
    while(1){
        short keyboardInput = *((short*)OSQPend(MSG_queue,
0, &error_code));
        alt_ucosii_check_return_code(error_code);

        if(keyboardInput == -1){
            *((int*)VIDEO_EFFECTS_0_BASE + 1) = reg1;
            reg1 = 0;
        }else{
            reg1 = reg1<<4 | keyboardInput;
            printf("%x\n", reg1);
        }
    }
}
```

Lectura de ficheros bitmap en SD

Formato de ficheros bitmap

- Fichero de mapa de bits
- Imágenes bidimensionales en color y B/N
- Son datos sin comprimir
- La cabecera tiene este formato

```
struct header{
char type[2];
unsigned long size;
char reserved[4];
unsigned long offset;

};
```

2 bytes con los caracteres “BM”
4 bytes que indican el tamaño total del archivo
4 bytes reservados

} 10 bytes

4 bytes que indican el offset desde el comienzo del archivo hasta el inicio del mapa de bits.

Cálculo del offset para obtener el inicio de los píxeles

Lectura bmp desde SD

- En el archivo los valores se organizan del byte menos significativo, al más significativo,
- La imagen se encuentra invertida: al inicio se encuentra la línea inferior, al final la línea superior.
- Los **píxeles se acomodan de izquierda a derecha: RGB será BGR**
- La longitud de línea en bytes múltiplo de 32.
- En caso de que la longitud de línea difiera de un múltiplo de 32, el programa de creación agrega los bytes faltantes a 0x00 para completar la longitud múltiplo de 32.

- Teniendo en cuenta esto ya se puede programar en C

Formato BMP

R	R	R	R	R	R	R	R	R	G
V	V	V	V	V	V	V	V	V	G
A	A	A	A	A	A	A	A	A	G
N	N	N	N	N	N	N	N	N	G
B	B	B	B	B	B	B	B	B	G

Figura 1. Imagen sintética de prueba.

R= Rojo	FF-00-00	RGB
V= Verde	00-FF-00	
A= Azul	00-00-FF	
N=Negro	00-00-00	
B=Blanco	FF-FF-FF	
G= Gris	7F-7F-7F	

Offset=0x36= 54 posiciones

cabecera

offset

B

G

R

42	4D	D6	00	00	00	00	00	00	00	36	00	00	00	28	00
00	00	0A	00	00	00	05	00	00	00	01	00	18	00	00	00
00	00	A0	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
FF	7F	7F	7F	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	7F	7F	7F	00	00	FF	00	00	FF	00	00	FF	00	00	FF
00	00	FF	00	00	FF	00	00	FF	00	00	FF	00	00	FF	00
00	7F	7F	7F	00	00	00	FF	00	00	FF	00	00	FF	00	00
FF	00	00	FF	00	00	FF	00	00	FF	00	00	FF	00	00	FF
00	7F	7F	7F	00	00	00	00	FF	00	00	FF	00	00	FF	00
00	FF	00	00	FF	00	00	FF	00	00	FF	00	00	FF	00	00
FF	7F	7F	7F	00	00										

Figura 2. Volcado de archivo de imagen de prueba

Lectura BMP SD

```
unsigned char cabecera [54];
void task_read_sd(void* pdata)
{
    int img_num = *(SW_switch_ptr) & 0xf;
    int* img_msg;
    INT8U error_code = OS_NO_ERR;
    while (1)
    {
        alt_up_sd_card_dev *device_reference = NULL;
        int connected = 0;

        device_reference = alt_up_sd_card_open_dev(SD_CARD_NAME);
        if (device_reference != NULL) {
                if ((connected == 0) && (alt_up_sd_card_is_Present())) {
                    printf("Card connected.\n");
                    if (alt_up_sd_card_is_FAT16()) {
                        printf("FAT16 file system detected.\n");
                        char file_name[] = "";
                        sprintf(file_name, "imagen%02u.bmp", img_num);
                        short int foto = alt_up_sd_card_fopen(file_name, false);
```

- Lectura de fichero de bmp identificado en SW
- Chequeos del HAL típicos con SD card
- OPEN_DEV
- Is_Present
- FAT16 o FAT32
- OPEN
- Uso de Task

Lectura BMP

- 10 bytes de cabecera
- 4 bytes de cálculo de offset
- Lectura de bytes hasta alcanzar zona de píxeles
- Lento como el caballo del “malo”… no leen la sd en bloque de 512bytes.
- Mejor uso de función hal de lectura de bloque y post procesado.

```
if(foto >= 0){
  for (int i = 0; i < 10; ++i) {
      cabecera[i] = alt_up_sd_card_read(foto);
  }
  int offset_array = 0;
  for (int i = 0; i < 4; ++i) {
      int offset_byte = alt_up_sd_card_read(foto);
      cabecera [i + 10] = (unsigned char)offset_byte;
      offset_array = (offset_byte << i*8) | offset_array;
  }
  printf("Offset %x\n", offset_array);
  offset_array-=14;
  printf("%x\n", offset_array);
  for (int i = 0; i < offset_array; ++i) {
      cabecera [i + 14] = alt_up_sd_card_read(foto);
 }
```

Lectura BMP (píxeles RGB 24 bits)

- Imagen de 240x400 pixeles 24 bits
- Formato RGB almacenado BGR en cada píxel
- Offset de imagen en formato X-Y
- Cierre de task con actualización del valor de la imagen desde MailBox

```
for (int row = 239; row >= 0; row--) {
        for (int col = 0; col < 400; col++) {
             short int blue = alt_up_sd_card_read(foto);
             short int green = alt_up_sd_card_read(foto);
             short int red = alt_up_sd_card_read(foto);
             blue = blue>>3;
             green = green>>2;
             red = red>>3;
             short pixel = (red<<11) | (green<<5) | (blue);
             int offset = (row << 9) + col;
             *(sd_card_buffer + offset) = pixel;
         }
}
alt_up_sd_card_fclose(foto);
```

```
        img_msg = (int*)OSMboxPend(MSG_box, 0, &error_code);
        alt_ucosii_check_return_code(error_code);
        img_num = *img_msg;
    }
```

Escritura SD de BMP

- Se prepara la cabecera de 54 bytes
- Mostramos solo la creación del fichero de imagen

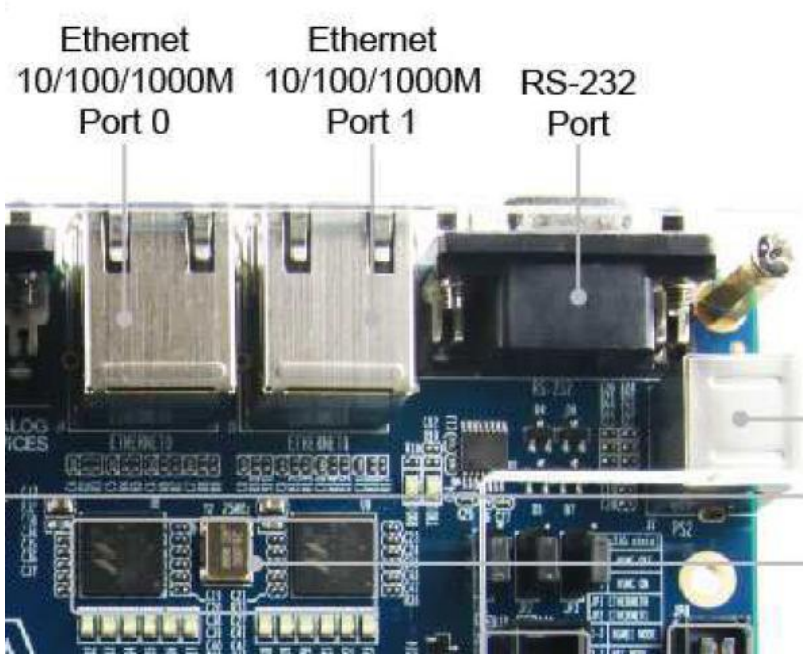
```
if(foto >= 0){
  for (int i = 0; i < 54; ++i) {
    alt_up_sd_card_write(foto, cabecera[i]);
  }
  for (int row = 239; row >= 0; row--) {
      for (int col = 0; col < 400; col++) {
          int offset = (row << 9) + col;
          short pixel = *(mtl_buffer + offset);
          unsigned char blue = (pixel & 0x1f)<<3;
          unsigned char green = ((pixel>>5) & 0x3f)<<2;
          unsigned char red = ((pixel>>11) & 0x1f)<<3;

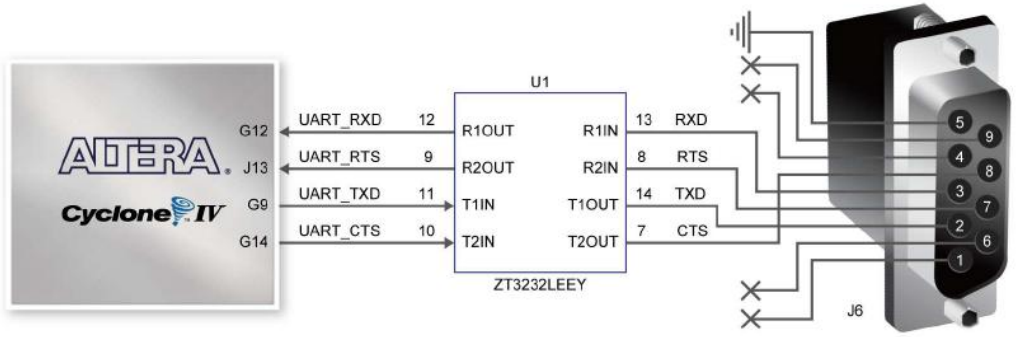
          if(alt_up_sd_card_write(foto, blue) == false){
                printf("Error blue %x\n", blue);
          }
          if(alt_up_sd_card_write(foto, green) == false){
                printf("Error green %x\n", blue);
          }
          if(alt_up_sd_card_write(foto, red) == false){
                printf("Error red %x\n", blue);
          }
      }
   }
  alt_up_sd_card_fclose(foto);
  printf("Guardado\n");
```

Uso de la UART

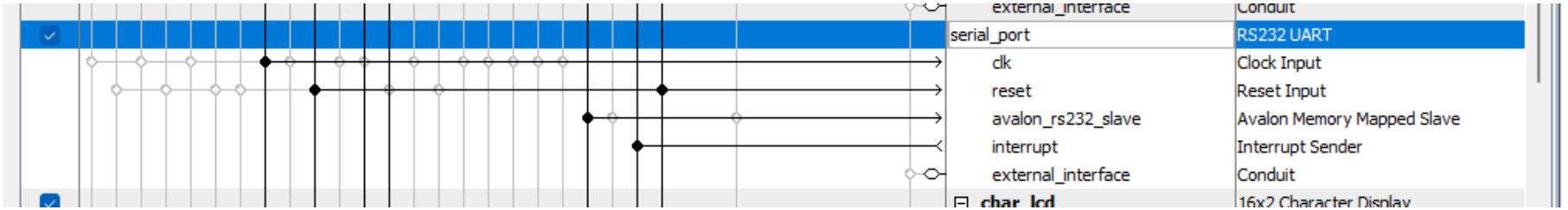
UART serie

- La UART es el puerto serie disponible en la placa DE2-115.
- Nos puede venir muy bien para comunicarnos con un PC sin usar el JTAG.
- Ello amplía mucho las capacidades de comunicar datos y ficheros desde un PC.

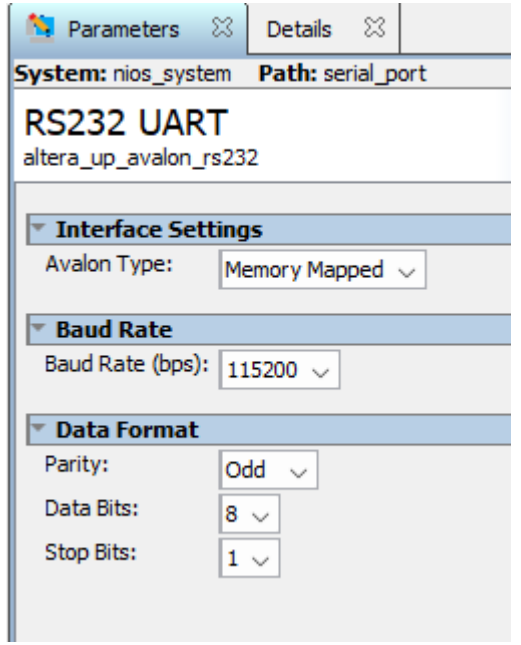




IP RS232



- DE2-115 Media Computer proporcionado por Terasic.
- Platform Designer IP.
- En *details* descargar documentación.
- En system.h direcciones y defines.



RS232 UART

Name	altera_up_avalon_rs232
Version	17.1
Author	Intel FPGA University Program
Description	RS232 UART Controller
Group	University Program/Communications
Data Sheet	file:///C:/intelFPGA_lite/17.1/ip/altera/university_program/communication/altera_up_avalon_rs232/doc/RS232.pdf
User Guide	file://ip/altera/university_program/memory/altera_up_avalon_sram/doc/SRAM_Controller.pdf
Release Notes	https://documentation.altera.com/#/link/hco1421698042087/hco1421698013408

```
538 #define ALT_MODULE_CLASS_serial_port altera_up_avalon_rs232
539 #define SERIAL_PORT_BASE 0x10000080
540 #define SERIAL_PORT_IRQ 10
541 #define SERIAL_PORT_IRQ_INTERRUPT_CONTROLLER_ID 0
542 #define SERIAL_PORT_NAME "/dev/serial_port"
543 #define SERIAL_PORT_SPAN 8
544 #define SERIAL_PORT_TYPE "altera_up_avalon_rs232"
```

RS232 UART Programming Model

- Register map

Table 2. RS232 UART Core register map

Offset in bytes	Register Name	R/W	Bit description										
			31...24	23...16	15	14...11	10	9	8	7	6...2	1	0
0	data	RW	*(1)*	RAVAIL	RVALID	*(1)*		PE	*(2)*	*(2)*	DATA		
4	control	RW	*(1)*	WSPACE	*(1)*			WI	RI	*(1)*		WE	RE

Table 3. Data register bits

Bit number	Bit/Field name	Read/Write	Description
8...0	DATA	R/W	The value to transfer to/from the RS232 UART Core. When writing, the DATA field is a character to be written to the write FIFO. When reading, the DATA field is a character read from the read FIFO.
9	PE	R	Indicates whether the DATA field had a parity error.
15	RVALID	R	Indicates whether the DATA and PE fields contain valid data.
23...16	RAVAIL	R	The number of characters remaining in the read FIFO (including this read).

Table 4. Control register bits

Bit number	Bit/Field name	Read/Write	Description
0	RE	R/W	Interrupt-enable bit for read interrupts
1	WE	R/W	Interrupt-enable bit for write interrupts
8	RI	R	Indicates that the read interrupt is pending
9	WI	R	Indicates that the write interrupt is pending
23...16	WSPACE	R	The number of spaces available in the write FIFO.

RS232 UART Programming Model

- Device Driver (HAL UART)

```
#include <stdio.h>
#include "includes.h"
#include <string.h>
#include "sys/alt_irq.h"
#include "altera_up_avalon_rs232.h"
```



alt_up_rs232_enable_read_interrupt
alt_up_rs232_disable_read_interrupt
alt_up_rs232_check_parity
alt_up_rs232_get_used_space_in_read_FIFO
alt_up_rs232_get_available_space_in_write_FIFO
alt_up_rs232_write_data
alt_up_rs232_read_data
alt_up_rs232_read_fd
alt_up_rs232_write_fd

```
void task3 (void* pdata)
{
    /* A simple program that recognizes the characters 't'  */

    char* msg = "Detected the character 't'.\n";
    char* msg2 = "Detected another character...\n";
    char* msg_idle = "RS232 line idle....write something please\n";
    char* mensaje ="";
```

UART RS232 Programming Model

- Podemos programar con stdio.h sin usar el hal (ANSI C).

```
/* alternativa utilizando stdio.h */
#ifdef RS232_STDIO
    FILE* fp;
    char prompt = 0;
    fp = fopen ("/dev/serial_port", "r+"); //Open file for reading and writing
    if (fp==NULL)
    {
        printf("Error en apertura de puerto serie\n");
    }else{
        printf("Puerto serie abierto\n");
    }

#endif
```

UART RS232. Task con polling X seg

```
while (1)
{
    if (fp!=NULL){
        prompt = getc(fp); // Get a character from the UART.
        printf("Has introducido el caracter hexadecimal %02x con valor ascii %c\n",(alt_8) prompt, prompt);

        if (prompt==0xFFFFFFFF){            // linea de entrada a nivel alto
            printf("Linea de entrada rs_232 idle\n");
            fwrite (msg_idle, strlen(msg_idle), 1, fp);
        }
        else if (prompt == 't')
        { // Print a message if character is 't'.
            fwrite (msg, strlen(msg), 1, fp);
            sprintf(mensaje, msg);
            printf(mensaje);
        }else{
            fwrite (msg2, strlen(msg2), 1, fp);
            sprintf(mensaje, msg2);
            printf(mensaje);
            printf("Character detected is %c\n",prompt);
        }

    }
    OSTimeDlyHMSM(0, 0, 3, 0);
}
```

Test con terminal en PC

```
    fp = fopen ("/dev/serial_port", "r+"); //Open file for reading and writing
    if (fp==NULL)
    {
        printf("Error en apertura de puerto serie\n");
    }else{
        printf("Puerto serie abierto\n");
    }

#endif

    while (1)
    {
        if (fp!=NULL){
            prompt = getc(fp); // Get a character from the UART.
            printf("Has introducido el caracter hexadecimal %02x con valor

            if (prompt==0xFFFFFFFF){            // linea de entrada a nive
                printf("Linea de entrada rs_232 idle\n");
                fwrite (msg_idle, strlen(msg_idle), 1, fp);
            }
            else if (prompt == 't')
            { // Print a message if character is 't'.
                fwrite (msg, strlen(msg), 1, fp);
                sprintf(mensaje, msg);
                printf(mensaje);
            }else{
```

Problems | Tasks | Console | Nios II Console | Properties

Hello_RTOS Nios II Hardware configuration - cable: USB-Blaster on localhost [USB-0] device ID: 1 instance ID: 0 name: jtaguart_0

```
Hello from task1
Hello from task2
Hello from task1
Hello from task2
Hello from task1
Hello from task2
Has introducido el caracter hexadecimal ffffffff con valor ascii ÿ
Linea de entrada rs_232 idle
Hello from task1
Hello from task2
```

Termite 3.4 (autor: CompuPhase)

COM6 115200 bps, 8O1, sin control de flujo | Config. | Limpiar | Sobre: | Cerrar

```
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
```

Usar UART como terminal

- Editar *system.h*

```
//#define ALT_STDOUT "/dev/jtag_uart"
#define ALT_STDOUT "/dev/serial_port"
```

```
        fwrite (msg2, strlen(msg2), 1, fp);
        sprintf(mensaje, msg2);
        printf(mensaje);
#endif
    }

}

/*
        fprintf(fp, "Closing the UART file.\n");
        fclose (fp);
*/

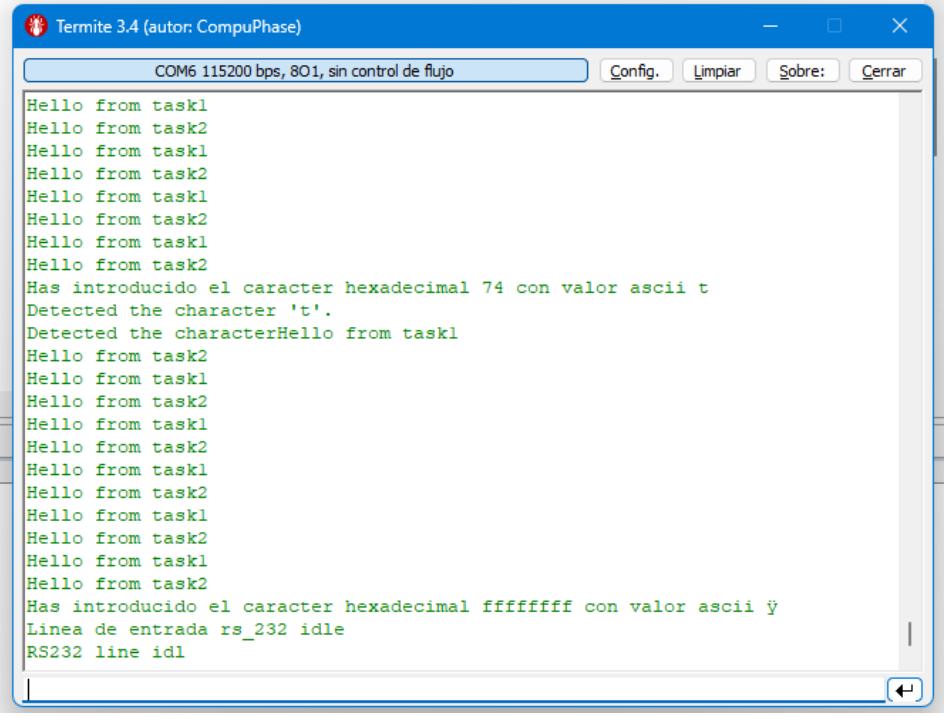
/* The main function creates the tasks and starts multi-tasking */
```

ems | Tasks | Console | Nios II Console | Properties

S Nios II Hardware configuration - cable: USB-Blaster on localhost [USB-0] device ID: 1 instance ID: 0 name: jtaguart_0

Convertimos la consola de NIOS en la UART





UART: Problemas asociados Task Polling

```
220     }
221
222 #endif
223
224     while (1)
225     {
226         if (fp!=NULL){
227             prompt = getc(fp); // Get a character from the UART.
228             printf("Has introducido el caracter hexadecimal %02x con valor
229
230             if (prompt==0xFFFFFFFF){            // linea de entrada a nive
231                 printf("Linea de entrada rs_232 idle\n");
232                 fwrite (msg_idle, strlen(msg_idle), 1, fp);
233             }
234             else if (prompt == 't')
235             { // Print a message if character is 't'.
236                 fwrite (msg, strlen(msg), 1, fp);
237                 sprintf(mensaje, msg);
238                 printf(mensaje);
239             }else{
```

Problems | Tasks | Console | Nios II Console | Properties

Hello_RTOS Nios II Hardware configuration - cable: USB-Blaster on localhost [USB-0] device ID: 1 instance ID: 0 name: jtaguart_0

```
Detected another character...
Character detected is d
Hello from task1
Hello from task2
Hello from task1
Hello from task2
Hello from task1
Hello from task2
Hello from task1
Hello from task2
```

Termite 3.4 (autor: CompuPhase)

COM6 115200 bps, 8O1, sin control de flujo | Config. | Limpiar | Sobre: | Cerrar

```
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
RS232 line idle....write something please
Detected another character...
Detected another character...
Detected another character...
RS232 line idle....write something please
Detected another character...
```

dead

Uso de UART con interrupciones

- Uso de ChatGPT

MP

program in C to send and receive data from UART peripheral in a NIOS soft processor in a Intel FPGA, the program uses ISR routine to get characters from PC serial terminal

Sure! Here's an example program in C that demonstrates sending and receiving data from a UART peripheral in a NIOS soft processor on an Intel FPGA. The program utilizes an ISR (Interrupt Service Routine) to receive characters from a PC serial terminal:

c Copy code

```c
#include <stdio.h>
#include <altera_avalon_uart_regs.h>
#include <altera_avalon_uart.h>
#include <sys/alt_irq.h>
#include "system.h"

#define UART_BASE       UART_0_BASE
#define IRQ             UART_0_IRQ

volatile int rx_flag = 0;
volatile char received_char;

void uart_isr(void* context, alt_u32 id)
{
    /* Read the received character from the UART data register */
    received_char = IORD_ALTERA_AVALON_UART_RXDATA(UART_BASE);
    rx_flag = 1;
}

int main()
{
    /* Initialize the UART */
    alt_up_uart_dev* uart_d
    alt_irq_register(IRQ, NU
```

Regenerate response

dev(UART_NAME);

UART con interrupciones con ChatGPT

```
#include <stdio.h>
#include <altera_avalon_uart_regs.h>
#include <altera_avalon_uart.h>
#include <sys/alt_irq.h>
#include "system.h"

#define UART_BASE       UART_0_BASE
#define IRQ             UART_0_IRQ

volatile int rx_flag = 0;
volatile char received_char;

void uart_isr(void* context, alt_u32 id)
{
   /* Read the received character from the UART data register */
   received_char = IORD_ALTERA_AVALON_UART_RXDATA(UART_BASE);
   rx_flag = 1;
}
```

```
int main()
{
   /* Initialize the UART */
   alt_up_uart_dev* uart_dev =
alt_up_uart_open_dev(UART_NAME);
   alt_irq_register(IRQ, NULL, uart_isr);
   alt_irq_enable(IRQ);

   printf("UART Test\n");

while (1) {
      if (rx_flag) {
         /* Echo back the received character */
         printf("Received: %c\n", received_char);
         rx_flag = 0;
      }

      /* Check for user input from the PC serial terminal */
      if (kbhit()) {
         char input = getchar();
         /* Send the character to the UART transmit buffer */
         IOWR_ALTERA_AVALON_UART_TXDATA(UART_BASE, input);
      }
   }

   return 0;
}
```

Sugerencias desde ChatGPT

Please note that this code assumes you have the necessary header files and libraries (**altera_avalon_uart_regs.h**, **altera_avalon_uart.h**, **sys/alt_irq.h**, and **system.h**) included and properly configured for your FPGA development environment.

Also, make sure to replace **UART_0_BASE** and **UART_0_IRQ** with the appropriate base address and IRQ number of your specific UART peripheral. The **UART_NAME** is a string that specifies the device name of your UART peripheral.

This code sets up an ISR (**uart_isr**) to handle incoming characters from the UART. When a character is received, it sets the **rx_flag** to indicate that a character has been received, and the main loop checks for this flag and echoes back the received character.

Additionally, the code checks for user input from the PC serial terminal using **kbhit()** and sends the input character to the UART transmit buffer.
Remember to customize the code according to your specific hardware configuration and requirements